00 · How to read this code

The project layout and the Python and TypeScript syntax you will meet in every other guide, explained once.

The big picture

HealthNexus has a **backend** (Python, FastAPI, PostgreSQL) and a **frontend** (TypeScript, React). The browser shows pages; pages ask the backend for data over HTTP; the backend checks who is asking, reads or writes the database, and answers with JSON.

Each feature guide (01 to 15) follows one feature from the screen to the database. This primer explains the building blocks those guides keep using, so the guides can focus on what each feature does.

Files involved

The main folders of the repository.

```
Health_Nexus/
├── backend/                    Python API (FastAPI)
│   ├── alembic/versions/       database migrations: how the tables were created, step by step
│   ├── app/
│   │   ├── main.py             creates the FastAPI app and mounts every route under /api/v1
│   │   ├── api/routes/*.py     endpoints, one file per area (vitals, alerts, phc, settings...)
│   │   ├── api/views.py        small helpers that turn database rows into responses
│   │   ├── services/*.py       the business rules (scoring, triage, catchment, visits...)
│   │   ├── models/*.py         one class per database table (SQLAlchemy)
│   │   ├── schemas/*.py        the shape of requests and responses (Pydantic)
│   │   ├── core/               settings, security helpers, "who is calling" checks
│   │   ├── db/session.py       the database connection
│   │   └── seeds/              scripts that fill a demo database
│   └── tests/                  pytest tests
├── frontend/
│   └── src/
│       ├── main.tsx            starts React and lists every page (the route table)
│       ├── pages/              one file per screen (citizen/, phc/, Settings, Profile...)
│       ├── components/         reusable pieces (app/ = inside the app, landing/, ui/)
│       ├── services/*.ts       one file per backend area: the only place that knows URLs
│       ├── lib/                helpers (useResource, params, format, routes, severity)
│       └── styles/             colours and fonts (CSS variables used by Tailwind classes)
└── ml/                         anomaly-model research and evaluation notebooks
```

How the pieces connect

1. A **page** (`frontend/src/pages/...`) is shown. It loads data with the `useResource` hook.
2. The page calls a function in `frontend/src/services/...`, for example `getSummary()`. That function sends an HTTP request with the shared axios client `api`, which adds the login token.
3. The request reaches a **route** function in `backend/app/api/routes/...`. FastAPI reads the URL, the query parameters and the JSON body, and runs the **dependencies** listed in the function signature first (who is calling, a database session).
4. The route uses **services** for the rules and **models** to read or write tables.
5. It returns a **schema** object; FastAPI turns it into JSON. The page receives it and React re-draws the screen.

Python syntax used in the backend

Code	What it means
`def f(x: int) -> str:`	A function. `x: int` and `-> str` are **type hints**: they say what goes in and out. Python does not enforce them, but FastAPI and Pydantic read them to validate data.
`async def` / `await`	An **asynchronous** function. `await db.execute(...)` pauses this request while the database works, so the server can handle other requests meanwhile.

Code	What it means
`str | None`	"A string, or None (nothing)". `None` is Python's empty value.
`@router.get("/summary")`	A **decorator**: registers the function below it as the handler for `GET /api/v1/<prefix>/summary`.
`user: User = Depends(require_user)`	FastAPI **dependency injection**: before the function runs, FastAPI calls `require_user`, which checks the login token and returns the user (or stops with 401/403).
`db: AsyncSession = Depends(get_session)`	Gives the function a database session for this request; it is closed automatically afterwards.
`Query(20, ge=1, le=100)`	A URL query parameter with default 20 that must be between 1 and 100 (`ge` = greater or equal, `le` = less or equal). Bad values are refused with 422 before the code runs.
`raise HTTPException(404, "...")`	Stops the request and answers with that HTTP status and message.
`class X(BaseModel):`	A **Pydantic** model: a typed record. Used for request bodies (validated on the way in) and responses (shaped on the way out).
`@dataclass(frozen=True)`	A simple record class; `frozen` means its fields cannot be changed after creation.
`select(A).where(A.x == 1)`	**SQLAlchemy** builds an SQL query in Python. Nothing runs until it is awaited: `await db.scalars(query)`.
`db.add(row)` / `await db.commit()`	Stage a new row, then save everything staged in one transaction. `flush()` sends it to the database early (to get its id) without finishing the transaction.
`[x * 2 for x in xs if x]`	A **list comprehension**: builds a new list in one line. `{k: v for ...}` builds a dictionary.
`f"Hello {name}"`	An **f-string**: text with values inserted.
`a if cond else b`	Inline if/else that gives a value.
`x or 0`	`x` if it has a value, otherwise 0.

TypeScript and React syntax used in the frontend

Code	What it means
`interface Reading { id: string }`	Describes the shape of an object. TypeScript checks the code uses it correctly; it disappears when the app is built.
`const f = (x: number) => x * 2;`	An **arrow function**. `const` declares a name that is not reassigned.
`function Page() { return <div/>; }`	A **React component**: a function that returns JSX (HTML-like markup). Its name starts with a capital letter.
`<Card title="Alerts">...</Card>`	**JSX**: using a component like an HTML tag. `title` is a **prop** (an input to the component).
`{ items.map((x) => <li key={x.id}>{x.name}</li>) }`	Curly braces put JavaScript inside JSX. `.map` turns a list into a list of elements; `key` tells React which item is which.
`{cond && <p>..</p>}` / `{a ? <A/> : <B/>}`	Show something only if `cond` is true / choose between two things.
`const [v, setV] = useState(0)`	A **state** variable. Calling `setV(1)` stores the new value and re-draws the component.
`useEffect(() => {...}, [a, b])`	Run code after drawing, again whenever `a` or `b` change. The function it returns runs as clean-up (for example, to cancel a timer).
`useMemo(() => calc, [deps])` / `useCallback`	Remember a computed value / a function between re-draws, recalculating only when `deps` change.
`useRef(x)`	A box whose `.current` value survives re-draws without causing one.
`const { data, error } = obj`	**Destructuring**: take named fields out of an object (or `[a, b] = list` out of a list).

Code	What it means
`a?.b` / `a ?? b` / `a ??= b`	Optional chaining: `undefined` instead of crashing if `a` is empty / use `b` if `a` is null or undefined / assign `b` only if `a` is empty.
`Hi ${name}`	A **template literal**: text with values inserted.
`async () => { try { await f() } catch (e) {} }`	Wait for a request; `catch` handles a failure.
`Promise.all([a(), b()])`	Run two requests at the same time and wait for both.
`className="rounded-xl text-sm"`	**Tailwind CSS**: each class is one small style (rounded corners, small text). Colour names such as `text-ink` come from `styles/tokens.css`.
`function f<T>(x: T): T`	A **generic**: works for any type `T`, keeping it known (for example `useResource<Summary>`).

Two helpers you will see on almost every page

- `useResource(fetcher, key)` (`frontend/src/lib/useResource.ts`) loads data for a page and returns `{ data, error, status, loading, reload }`. It re-loads when `key` changes and ignores answers that arrive after a newer request. Walkthrough in guide 14.
- `useQueryState()` (`frontend/src/lib/params.ts`) keeps filters in the URL (`?days=30&page=2`), so a filtered page can be bookmarked or shared. Values from the URL are always checked with `oneOf`, `intParam` or `textParam` before use.

Words used in the guides

Word	Meaning
Citizen	A person who logs readings (role `user`).
PHC worker	Staff at a Primary Health Centre who reviews flags and visits people (role `phc_worker`).
Reading	One set of vitals logged at one moment: a row in `health_records`.
Flag	A reading the anomaly model called unusual: `processed_data.is_anomaly = true`. **Open** until a worker reviews or dismisses it.
Catchment	The citizens linked to the same health centre as a worker (same `phc_id`). A worker only ever sees their catchment.
Baseline	A person's own usual value for a vital, learned from their earlier readings.

Guides in this set

Guide	Feature
01	Log vitals and the anomaly model
02	Citizen dashboard
03	History
04	Alerts, notifications and badges
05	PHC dashboard and the catchment rule
06	Worklist and reviewing flags
07	Citizens directory
08	Citizen record
09	Visits and schedule
10	Reports
11	Risk map

Guide	Feature
12	Settings
13	Profile
14	Foundations: app setup, database, data loading, shared UI
15	Landing page

Auth (login, sessions) and onboarding are covered by the separate **Auth and Onboarding Guide**.